

TEORI BAHASA DAN AUTOMATA

Modul Pembelajaran: Deterministic Finite Automata (DFA)

Modul komprehensif mengenai konsep dasar, definisi formal, struktur loop, dan contoh perancangan DFA.

Contents

1	Konsep Dasar Deterministic Finite Automata (DFA)	2
1.1	Pengantar dan Prinsip Kerja DFA	2
1.2	Struktur Diagram Transisi DFA	2
1.3	Syarat Diterima dan Ditolaknya String	2
2	Definisi Formal DFA	4
2.1	Tabel Transisi	4
2.2	Fungsi Transisi Diperluas dan Bahasa yang Diterima	4
3	Loop Transisi dan Dead State	6
3.1	Makna dan Karakteristik Loop	6
3.2	Dead State / Trap State	6
4	Contoh Komprehensif Perancangan dan Simulasi DFA	7
4.1	Contoh 1: DFA Minimal Satu Simbol "1"	7
4.2	Contoh 2: DFA String Dimulai dengan "0"	7
4.3	Contoh 3: DFA Panjang String Persis 2	8
4.4	Contoh 4: Evaluasi Pola String yang Rumit	8

1 Konsep Dasar Deterministic Finite Automata (DFA)

1.1 Pengantar dan Prinsip Kerja DFA

Deterministic Finite Automaton (DFA) adalah mesin abstrak yang digunakan untuk mengenali pola atau bahasa reguler[cite: 1, 3]. Mesin ini beroperasi dengan membaca sekumpulan karakter (*String*) dari sebuah pita masukan (*Input Tape*) satu per satu dari kiri ke kanan[cite: 1]. Hasil akhir dari pemrosesan ini hanya memiliki dua kemungkinan: mesin menerima (*Accept*) string tersebut, atau menolaknya (*Reject*)[cite: 1].

Sifat fundamental dari DFA adalah **Determinisme**[cite: 1]. Artinya, jika mesin berada di suatu *state* dan menerima sebuah simbol input tertentu, maka arah perpindahan *state* selanjutnya sudah dapat dipastikan secara mutlak[cite: 1]. Tidak ada keraguan atau percabangan ganda untuk satu simbol input yang sama[cite: 1]. Misalnya, jika mesin berada di *state* A dan membaca angka "1", ia hanya memiliki satu jalan, yaitu menuju *state* B[cite: 1].

Karakteristik Determinisme pada DFA

- **Kepastian Arah:** Hanya mempunyai tepat satu *state* tujuan untuk setiap kombinasi *state* asal dan input[cite: 1].
- **Kewajiban Transisi:** Setiap *state* di dalam DFA **wajib** memiliki transisi keluar untuk setiap simbol yang ada di dalam alfabet (himpunan input)[cite: 1].

1.2 Struktur Diagram Transisi DFA

Secara visual, DFA digambarkan menggunakan graf berarah yang disebut diagram transisi. Terdapat lima elemen visual utama dalam diagram ini:

1. **Initial State:** Merupakan *state* awal tempat mesin mulai membaca string, ditandai dengan anak panah yang masuk tanpa *state* asal[cite: 1].
2. **State:** Merepresentasikan kedudukan atau memori mesin pada suatu waktu, digambarkan dengan lingkaran[cite: 1].
3. **Transition:** Arah perpindahan antar *state*, digambarkan dengan garis berpanah[cite: 1].
4. **Input:** Simbol yang dituliskan di atas garis transisi, bertindak sebagai pemicu perpindahan *state*[cite: 1].
5. **Final State:** Kedudukan akhir penerimaan, digambarkan dengan lingkaran ganda (*double circle*)[cite: 1]. Sebuah DFA dapat memiliki lebih dari satu *final state* (*multi final state*)[cite: 1].

1.3 Syarat Diterima dan Ditolaknya String

Proses membaca input disebut *scanning*. Head mesin akan membaca satu per satu karakter hingga pita kosong (*empty tape* atau *input finished*)[cite: 1].

- **Diterima (Accept):** String diterima jika dan hanya jika semua input string telah selesai diperiksa dan mesin berhenti tepat pada salah satu *final state*[cite: 1].

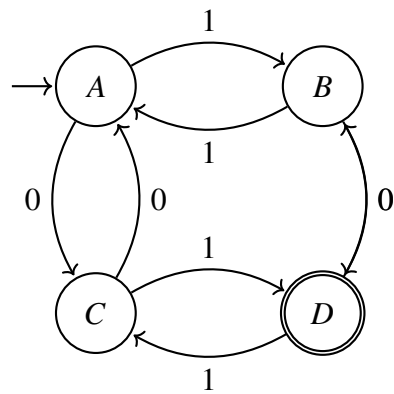


Figure 1: Contoh Struktur Diagram Transisi DFA

- **Ditolak (Reject):** String ditolak jika setelah semua input habis dibaca, mesin berhenti di state yang **bukan** merupakan *final state*[cite: 1].

2 Definisi Formal DFA

Untuk menganalisis DFA secara matematis, kita menggunakan model 5-tupel. Secara formal, Deterministic Finite Automaton M didefinisikan sebagai:

$$M = (Q, \Sigma, \delta, q_0, F)$$

[cite: 3]

Berikut adalah penjelasan komprehensif dari setiap komponen tupel tersebut:

1. **Q (Himpunan State):** Merupakan himpunan hingga dari semua state atau kedudukan yang ada di dalam mesin[cite: 3].
2. **Σ (Input Alphabet):** Himpunan hingga dari simbol-simbol input yang dikenali oleh mesin[cite: 3]. Harap diingat bahwa string kosong (epsilon / ϵ) tidak pernah masuk ke dalam anggota himpunan alfabet ini ($\epsilon \notin \Sigma$)[cite: 3].
3. **δ (Fungsi Transisi):** Sebuah fungsi yang mendefinisikan perpindahan state, dipetakan sebagai $\delta : Q \times \Sigma \rightarrow Q$ [cite: 3]. Artinya, fungsi ini mengambil satu state asal dan satu simbol input, kemudian memetakan (menghasilkan) tepat satu state tujuan[cite: 3].
4. **q_0 (State Awal):** State di mana komputasi selalu dimulai. Syarat mutlaknya adalah $q_0 \in Q$ [cite: 3].
5. **F (Himpunan Final State):** Himpunan state penerima. Karena final state adalah bagian dari state yang ada, maka $F \subseteq Q$ [cite: 3].

2.1 Tabel Transisi

Fungsi transisi δ sangat ideal jika direpresentasikan dalam bentuk tabel (Tabel Transisi)[cite: 3]. Baris merepresentasikan state asal, dan kolom merepresentasikan simbol input[cite: 3]. Persilangan antara baris dan kolom adalah state tujuan[cite: 3].

Misalkan kita memiliki DFA dengan $Q = \{q_0, q_1, q_2\}$, $\Sigma = \{a, b\}$, dan $F = \{q_0, q_1\}$. Berikut adalah bentuk tabel transisinya[cite: 3]:

δ	a	b
$\rightarrow *q_0$	q_0	q_1
$*q_1$	q_0	q_2
q_2	q_2	q_2

Table 1: Tabel Transisi. Tanda panah ($\rightarrow$) menandakan state awal, dan asterisk (*) menandakan final state.

2.2 Fungsi Transisi Diperluas dan Bahasa yang Diterima

Fungsi transisi δ biasa hanya membaca satu karakter. Untuk membaca kumpulan karakter (string), digunakan **fungsi transisi diperluas** yang disimbolkan dengan δ^* [cite: 3].

$$\delta^* : Q \times \Sigma^* \rightarrow Q$$

[cite: 3] $\delta^*(q, w) = q'$ menggambarkan bahwa dari state q , jika diberikan rentetan string w , maka akan berakhir di state q' [cite: 3].

Bahasa yang Diterima ($L(M)$): Adalah himpunan dari seluruh string yang berakhir di state final[cite: 3].

$$L(M) = \{w \in \Sigma^* \mid \delta^*(q_0, w) \in F\}$$

[cite: 3]

Bahasa yang Ditolak ($\overline{L(M)}$): Adalah himpunan dari seluruh string yang berakhir di state bukan final[cite: 3].

$$\overline{L(M)} = \{w \in \Sigma^* \mid \delta^*(q_0, w) \notin F\}$$

[cite: 3]

3 Loop Transisi dan Dead State

Sebuah **Loop Transisi** terjadi ketika fungsi transisi mengarahkan kembali ke state asalnya, yaitu $\delta(q_i, a) = q_i$ [cite: 2]. Saat mesin mengeksekusi loop, mesin membaca simbol pada input tape dan menggeser head, namun posisi state tidak berubah[cite: 2].

3.1 Makna dan Karakteristik Loop

Keberadaan loop bukan berarti mesin tersangkut, melainkan menunjukkan desain yang efisien dan stabil[cite: 2]. Ada tiga karakteristik penting dari loop transisi:

1. **Satu Transisi per Simbol:** Loop membantu mesin untuk tetap mematuhi aturan deterministik bahwa setiap simbol harus memiliki transisi, sehingga loop melengkapi arah transisi yang tidak menyebabkan perubahan logika[cite: 2].
2. **Tidak Melanggar Determinisme:** Output dari loop bersifat tunggal dan pasti, sehingga sifat deterministik tetap valid[cite: 2].
3. **Representasi Stabilitas:** Menandakan bahwa setelah mencapai suatu kondisi, mesin tidak berubah logika saat membaca simbol tertentu[cite: 2].

Contoh Analisis: DFA Paritas (Jumlah 0 Genap)

Pada mesin yang bertugas menghitung apakah jumlah 0 bernilai genap, simbol "1" akan selalu menghasilkan loop di state mana pun[cite: 2]. Ini dikarenakan angka "1" bersifat netral dan tidak mempengaruhi penghitungan paritas genap/ganjil dari angka "0"[cite: 2]. Loop pada angka 1 memastikan bahwa status genap tetap genap, dan ganjil tetap ganjil[cite: 2].

3.2 Dead State / Trap State

Bentuk paling ekstrem dari loop adalah **Trap State** (atau *Dead State*)[cite: 2, 3]. Trap state adalah sebuah state (yang *bukan* final state) di mana **semua** input dari alfabet akan membuat mesin melakukan loop kembali ke state tersebut[cite: 2].

$$\delta(q_t, 0) = q_t \quad \text{dan} \quad \delta(q_t, 1) = q_t$$

[cite: 2]

- Jika mesin masuk ke trap state, mesin tidak akan pernah bisa keluar[cite: 2].
- Fungsi trap state adalah "menyerap" semua sisa input yang ada pada pita[cite: 2].
- Karena trap state bukan final state, maka string apa pun yang pernah menyentuh trap state dapat dipastikan 100% ditolak[cite: 2].

4 Contoh Komprehensif Perancangan dan Simulasi DFA

Berikut adalah beberapa contoh perancangan mesin DFA berserta dengan simulasi fungsi transisi diperluasnya.

4.1 Contoh 1: DFA Minimal Satu Simbol "1"

Misalkan $L(M) = \{w \in \{0, 1\}^* \mid w \text{ mengandung minimal satu } 1\}$ [cite: 2].

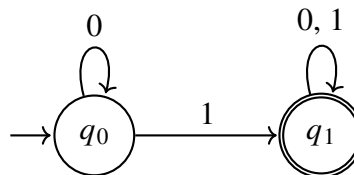


Figure 2: DFA Mengandung Minimal Satu "1"

Simulasi Input $w = 0011$ (**Diterima**) [cite: 2]:

- $\delta(q_0, 0) = q_0$ (Loop pencarian, belum menemukan 1) [cite: 2].
- $\delta(q_0, 0) = q_0$ (Loop pencarian) [cite: 2].
- $\delta(q_0, 1) = q_1$ (Berpindah, "1" ditemukan) [cite: 2].
- $\delta(q_1, 1) = q_1$ (Loop stabilitas, tetap di accept state) [cite: 2].
- Karena berakhir di $q_1 \in F$, maka string **Diterima** [cite: 2].

4.2 Contoh 2: DFA String Dimulai dengan "0"

Misalkan $L(M) = \{0, 00, 01, 000, 010, \dots\}$ [cite: 3]. Ini berarti huruf pertama *wajib* 0. Jika huruf pertamanya 1, maka otomatis gagal selamanya.

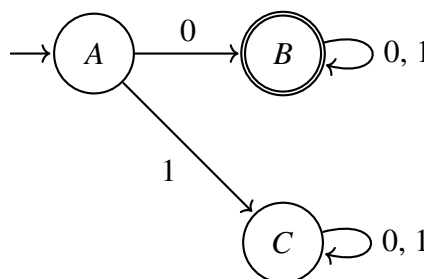


Figure 3: DFA Dimulai dengan 0. State C bertindak sebagai Dead State.

Simulasi Input $w = 101$ (**Ditolak**) [cite: 3]:

- $\delta(A, 1) = C$ (Huruf pertama salah, masuk ke Dead State) [cite: 3].
- $\delta(C, 0) = C$ (Terjebak di Dead State) [cite: 3].
- $\delta(C, 1) = C$ (Terjebak di Dead State) [cite: 3].
- Karena berakhir di $C \notin F$, maka string **Ditolak** [cite: 3].

4.3 Contoh 3: DFA Panjang String Persis 2

Misalkan $L(M) = \{00, 01, 10, 11\}$, dengan alfabet $\Sigma = \{0, 1\}$ [cite: 3].

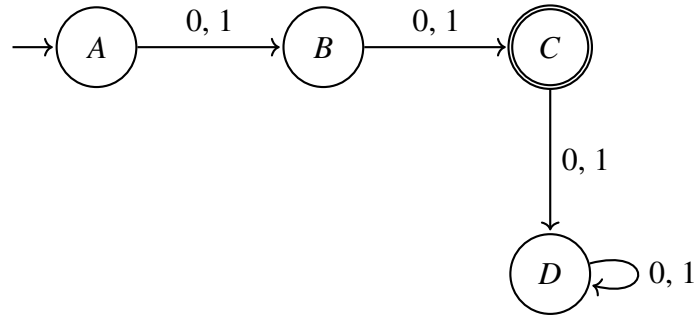


Figure 4: DFA Panjang Tepat 2. State D bertindak sebagai Dead State.

Pada kasus ini, langkah komputasinya sangat linier. Mesin hanya menerima string jika setelah membaca tepat dua karakter, mesin berhenti[cite: 3]. Jika ada karakter ketiga yang dibaca, mesin akan langsung pindah ke state D (*Dead State*) dan menolak string tersebut selamanya[cite: 3].

4.4 Contoh 4: Evaluasi Pola String yang Rumit

Kita uji DFA dengan fungsi perluasan $\delta^*(q_0, w)$ menggunakan definisi pada tabel transisi Contoh 3 sebelumnya (dengan state q_0, q_1, q_2)[cite: 3]. Alfabet $\Sigma = \{a, b\}$, $F = \{q_0, q_1\}$.

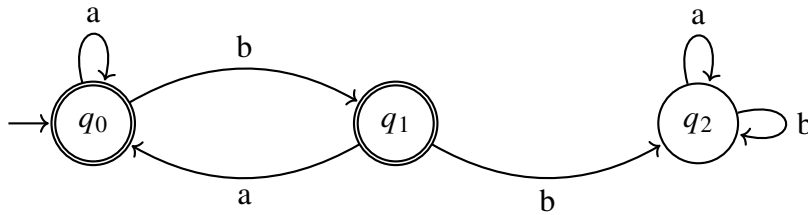


Figure 5: DFA Pola Bahasa. State q_2 adalah Dead State.

Tracing Input $w = abababaa$ [cite: 3]: Langkah-langkah evaluasinya diselesaikan dari kiri ke kanan dengan menurunkan fungsi transisi yang diperluas:

1. $\delta^*(q_0, abababaa) \Rightarrow \delta(q_0, a)$ menghasilkan q_0 , sisa string: $bababaa$ [cite: 3].
2. $\Rightarrow \delta^*(q_0, bababaa) \Rightarrow \delta(q_0, b)$ menghasilkan q_1 , sisa string: $ababaa$ [cite: 3].
3. $\Rightarrow \delta^*(q_1, ababaa) \Rightarrow \delta(q_1, a)$ menghasilkan q_0 , sisa string: $babaa$ [cite: 3].
4. $\Rightarrow \delta^*(q_0, babaa) \Rightarrow \delta(q_0, b)$ menghasilkan q_1 , sisa string: $abaa$ [cite: 3].
5. $\Rightarrow \delta^*(q_1, abaa) \Rightarrow \delta(q_1, a)$ menghasilkan q_0 , sisa string: baa [cite: 3].
6. $\Rightarrow \delta^*(q_0, baa) \Rightarrow \delta(q_0, b)$ menghasilkan q_1 , sisa string: aa [cite: 3].
7. $\Rightarrow \delta^*(q_1, aa) \Rightarrow \delta(q_1, a)$ menghasilkan q_0 , sisa string: a [cite: 3].
8. $\Rightarrow \delta^*(q_0, a) \Rightarrow \delta(q_0, a)$ menghasilkan q_0 (String habis)[cite: 3].

Karena *tracing* berakhir tepat pada state q_0 dan $q_0 \in F$ (State AKHIR), maka kalimat *abababaa* dinyatakan **Diterima**[cite: 3]. Dapat disimpulkan bahwa bahasa $L(M)$ ini menerima string seperti *abababaa*, *aaabab*, *aabababa*, selama tidak menjumpai kombinasi substring "bb" yang memicu perpindahan ke dead state q_2 [cite: 3].